

# INTENTFIX: Automated Logic Vulnerability Repair via LLM-Driven Intent Modeling

Jinseok Heo  
Department of Electrical and  
Computer Engineering  
Sungkyunkwan University  
Suwon, Republic of Korea  
mrhjs225@skku.edu

Dongwook Choi  
Department of Computer Science and  
Engineering  
Sungkyunkwan University  
Suwon, Republic of Korea  
dwchoi95@skku.edu

Jinyoung Kim  
Department of Computer Science and  
Engineering  
Sungkyunkwan University  
Suwon, Republic of Korea  
danpoong@skku.edu

Misoo Kim  
Chonnam National University  
Gwangju, Republic of Korea  
misoo.kim@jnu.ac.kr

Eunseok Lee  
College of Computing and Informatics  
Sungkyunkwan University  
Suwon, Republic of Korea  
lees@skku.edu

## Abstract

Logic vulnerabilities, which arise from semantic gaps between a developer's intent and the actual code, represent a critical and growing challenge in software security. Unlike syntactic bugs, these vulnerabilities pass traditional testing while harboring critical security flaws that can lead to severe breaches. We introduce INTENTFIX, a novel framework that automatically repairs logic vulnerabilities through intent-centric security analysis. INTENTFIX first leverages a Large Language Model (LLM) to systematically extract and formalize the developer's implicit intent into a structured model. It then performs a differential analysis between this intent model and the implementation to precisely identify semantic gaps. Finally, it synthesizes and refines a patch through a multi-aspect, LLM-driven reasoning process.

We evaluated INTENTFIX on a comprehensive dataset of 1,107 real-world CVE cases across five vulnerability types and 19 programming languages, achieving a patch accuracy of 64.5%, a 1.97× improvement over strong Chain-of-Thought (CoT) LLM baselines.

This work makes three primary contributions: (1) We formalize intent-centric analysis as a new theoretical foundation for logic vulnerability repair. (2) We introduce a novel, structured framework that effectively harnesses an LLM's reasoning capabilities for security. (3) We provide extensive empirical evidence validating our approach and offering new insights into the role of context in automated program repair.

## CCS Concepts

• **Software and its engineering** → **Software maintenance tools**;  
*Software evolution*; *Maintaining software*.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

ICSE '26, Rio de Janeiro, Brazil

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2025-3/26/04

<https://doi.org/10.1145/3744916.3773260>

## Keywords

Logical vulnerabilities, Intent modeling, Large language model, Automated vulnerability repair, Business logic errors, Vulnerability detection

### ACM Reference Format:

Jinseok Heo, Dongwook Choi, Jinyoung Kim, Misoo Kim, and Eunseok Lee. 2026. INTENTFIX: Automated Logic Vulnerability Repair via LLM-Driven Intent Modeling. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3744916.3773260>

## 1 Introduction

The exponential growth in software complexity has fundamentally shifted the attack landscape from syntactic vulnerabilities to semantic design flaws [19, 24]. The OWASP Top 10 report highlights a significant shift towards design-level flaws, with categories like 'A04:2021-Insecure Design' underscoring that many critical security breaches stem from logic vulnerabilities [29]. These are flaws arising not from simple coding errors but from misalignments between intended behavior and actual implementation. These design-level flaws can have catastrophic real-world consequences. For example, the infamous 2016 DAO hack resulted in \$60 million losses due to a reentrancy vulnerability—a logic flaw where the contract's state update occurred after external calls, violating the intended transaction atomicity [1].

Logic vulnerabilities present unique challenges that render traditional security tools ineffective [40, 46]. Consider a typical e-commerce application where order cancellation should be restricted to order owners. The following code snippet illustrates this vulnerability:

```
@app.route('/orders/<id>/cancel', methods=['POST'])
def cancel_order(id):
    order = Order.query.get(id)
    order.status = 'cancelled' # Missing ownership check
    db.session.commit()
    return {'status': 'success'}
```

This vulnerability illustrates why traditional security tools are ineffective. Static analysis tools, lacking an understanding of domain-specific intentions, cannot flag the missing ownership check [23, 34].

Similarly, dynamic testing approaches struggle because there is no explicit test oracle to verify this implicit requirement [2].

Logic vulnerabilities represent a significant and growing threat to organizations, with data breach costs reaching unprecedented levels [12, 15]. Despite their prevalence and impact, state-of-the-art learning-based AVR techniques struggle with this class of vulnerabilities [4, 19]. For example, prominent models like VulRepair [8] and VulMaster [50] show significantly lower performance on logic flaws compared to syntactic bugs like memory safety issues. This highlights a fundamental limitation of approaches that lack a deep semantic understanding of program intent.

We propose a fundamental shift from *implementation-focused* to *intent-centric* vulnerability analysis. Our key insight is that logic vulnerabilities fundamentally arise from semantic gaps between what developers intend and what code actually implements. By explicitly modeling developer intentions and systematically comparing them against implementations, we can identify and repair these subtle yet critical vulnerabilities.

We present INTENTFIX, a framework that leverages a Large Language Model's semantic understanding capabilities to bridge this gap. Unlike existing approaches that rely on predefined patterns or test suites, INTENTFIX dynamically constructs an intent model by prompting an LLM to analyze not just the code itself, but also a holistic set of contextual clues such as variable names, function signatures, comments, and domain-specific terminology found within the codebase. This extracted intent model then serves as a security oracle for subsequent LLM-driven analysis and repair phases. Our framework introduces three key innovations: **(1) Structured Intent Modeling**, a formal representation of developer intentions as five-tuple models; **(2) LLM-Driven Semantic Analysis**, a systematic methodology for extracting implicit intentions using carefully designed prompt chains; and **(3) Self-Verifying Patch Synthesis**, an iterative patch generation process with multi-aspect refinement.

This paper makes the following contributions:

- **Novel Framework:** We introduce INTENTFIX, an automated framework that uses explicit intent modeling for logic vulnerability repair. It achieves 64.5% accuracy on 1,107 real-world CVEs, a  $1.97\times$  improvement over state-of-the-art CoT-prompted LLM(GPT-4.1-mini) baselines.
- **Theoretical Foundation:** We propose a structured 5-tuple intent model (Domain, Actors, Resources, Permissions, Semantic Constraints) that formalizes implicit security requirements, turning the ambiguous problem of logic vulnerability detection into a verifiable analysis of intent implementation gaps.
- **Empirical Insights:** Through extensive evaluation, we provide a deep analysis of our framework's performance, demonstrating its effectiveness across diverse vulnerability types and programming languages, and quantifying the contribution of each of its components.
- **Verifiability:** We release our framework, dataset, and experimental results publicly to ensure verifiability and facilitate future research in intent-centric software security.

## 2 Related Work

**LLM-Based Automated Vulnerability Repair** Recent advances in automated vulnerability repair (AVR) have shifted from pattern-based to LLM-powered approaches [48]. Early neural methods like VulRepair [8] achieved 35-45% repair rates but showed significantly lower performance on logic vulnerabilities (23%) compared to memory safety issues (61%). The seminal work by Xia and Zhang [43] demonstrated that pre-trained LLMs substantially outperform traditional APR techniques without specialized training. Recent studies have further explored zero-shot prompting strategies for security-related code repair, showing that prompt design significantly influences repair effectiveness [18].

In the broader program repair domain, industrial deployments validate the effectiveness of LLM-based approaches: Microsoft's InferFix [17] achieves 65.6% accuracy for C# in production, while agent-based systems like RepairAgent [3] fix complex bugs at \$0.14 per bug. Beyond industrial settings, automated repair techniques have also been adapted for educational contexts, where program diversification strategies improve repair rates for student code [6]. However, these general-purpose repair approaches excel primarily at well-defined patterns identifiable by static analyzers, struggling with the semantic gaps characterizing logic vulnerabilities.

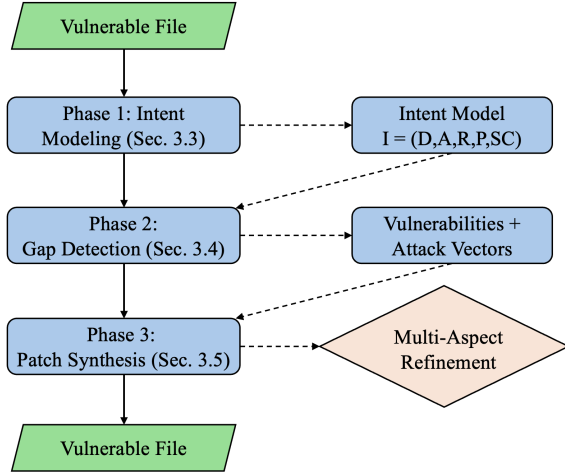
**Logic Vulnerability and Repair** Logic vulnerabilities, formally categorized as CWE-840 (Business Logic Errors) by MITRE [25], represent a class of security flaws that arise from incorrect implementation of business rules and application logic [19]. Unlike memory safety errors or injection vulnerabilities, logic flaws are semantic in nature—they exist in syntactically correct code that violates implicit security requirements [47]. Their potential for catastrophic impact is well-documented, as seen in major financial losses from flaws like reentrancy vulnerabilities [1].

Recent learning-based repair techniques still struggle with these semantic flaws [8]. The emergence of LLMs offers new possibilities [20, 34, 45], but these methods often rely on predefined patterns or manual scenario definitions, limiting their adaptability. With logic flaws becoming a major attack vector [13], the need for automated, intent-aware repair approaches is critical, motivating our work.

**Intent Modeling and Specification Inference.** While intent modeling has theoretical foundations in requirements engineering (e.g., KAOS [38], Problem Frames [14]), its systematic application to vulnerability repair remains unexplored. Recent automated approaches like SpecRover [31] extract function-level specifications, achieving 50% improvement on SWE-Bench, while InferROI [39] infers resource management intent for static analysis.

These works validate intent extraction's value but focus on functional specifications or specific vulnerability classes rather than comprehensive security properties. No existing work provides a structured framework for extracting and formalizing security-relevant intent across domains.

**Reasoning-Enhanced Repair Techniques.** Recent prompting advances significantly improve LLM performance on complex tasks [41]. VSP [27] uses data/control flow facts as "thoughts" in CoT prompting, achieving 553.3% higher F1 for vulnerability identification. Similarly, studies on code infilling have demonstrated that context representation choices—such as combining code with



**Figure 1: INTENTFIX Framework Architecture.** Solid arrows represent the main data flow through three sequential phases, while dashed arrows indicate feedback and refinement processes. Numbers in parentheses reference the corresponding sections.

function summaries—substantially affect model performance [9]. Self-debugging approaches [5] and iterative refinement [16] demonstrate that multi-step reasoning improves repair quality.

**Research Gap.** Our analysis reveals a critical gap: existing works lack explicit security intent modeling, rely on external oracles insufficient for logic flaws, and have limited semantic reasoning. INTENTFIX addresses this by making developer intent a first-class, verifiable artifact in the repair pipeline, shifting the paradigm from implementation-focused to intent-centric analysis.

### 3 INTENTFIX

We present INTENTFIX, a novel framework that automatically detects and repairs logic vulnerabilities by explicitly modeling developer intentions and identifying semantic gaps between these intentions and actual implementations. This section describes the theoretical foundations, system architecture, and technical details of our approach.

#### 3.1 Overview and Key Insights

INTENTFIX operates on three key insights:

- (1) **Intentions as Security Specifications:** Developer intentions, while implicit in code, serve as de facto security specifications that define correct behavior.
- (2) **LLMs as Semantic Reasoners:** Large Language Models can extract high-level intentions from code by understanding domain contexts, programming patterns, and security implications.
- (3) **Iterative Refinement:** Complex vulnerability repair requires multiple rounds of generation and refinement to ensure correctness across security, functionality, and quality dimensions.

#### 3.2 Conceptual Framework and Problem Formulation

##### 3.2.1 Formal Problem Definition.

**Definition 3.1 (Logic Vulnerability).** A logic vulnerability is a tuple  $V = (S, I, G)$  where:

- $S$  is the source code implementation
- $I$  is the intended behavior specification
- $G = \{g \mid g \in S \times I, \neg \text{satisfies}(S, I)\}$  represents semantic gaps

**Definition 3.2 (Intent Model).** An intent model  $I = (D, A, R, P, SC)$  is a formal representation of developer intentions:

- $D$  (Domain):  $\{name : string, entities : E[], context : string\}$  - Application domain and key entities
- $A$  (Actors):  $\{id : string, type : \{human, system, external\}, role : string, privileges : P[]\}$  - System actors and their roles
- $R$  (Resources):  $\{id : string, type : \{data, api, file\}, ops : O[], invariants : Inv[]\}$  - Protected resources
- $P$  (Permissions):  $\{actor : A, resource : R, allowed_ops : O[], conditions : C[]\}$  - Access control matrix
- $SC$  (Semantic Constraints):  $\{id : string, type : T, rule : \phi, scope : S, priority : int\}$  - Domain-specific security rules

Our 5-tuple intent model extends the well-established Attribute-Based Access Control (ABAC) model [11] and integrates concepts from threat modeling frameworks such as STRIDE [33] and PASTA [37]. While ABAC provides the foundation for subjects (Actors), objects (Resources), and policies (Permissions), we introduce two critical extensions specifically designed for logic vulnerability analysis:

- **Domain (D):** This component explicitly models the application's business context, which is essential for an LLM to correctly interpret the semantics of the code. For example, the meaning of a 'cancel' operation is entirely different in an 'e-commerce' domain versus a 'background job' domain.
- **Semantic Constraints (SC):** This is the most crucial extension for logic flaws. It moves beyond static permissions to capture dynamic, stateful, and context-dependent business rules (e.g., "an order can only be cancelled *if* its status is 'pending' and *within* 5 minutes of placement"). These constraints are the primary source of logic vulnerabilities.

Unlike prior specification-inference work that focuses on functional correctness, our 5-tuple model is tailored to security reasoning. First, it extends ABAC-style subject-object-policy structures with explicit *domain* and *semantic-constraint* dimensions that capture business rules and regulatory requirements. Second, it is instantiated via a hierarchical LLM pipeline that extracts  $D$ ,  $A$ ,  $R$ ,  $P$ , and  $SC$  using role-specific prompts and schema-constrained generation. Third, it is designed to be checked by an automated semantic-gap detection procedure that reasons about whether the implementation actually enforces constraints in  $SC$ . Together, these design choices turn the intent model into a reusable, implementation-agnostic abstraction for analyzing logic vulnerabilities.

Thus, our 5-tuple model is a purpose-built framework designed not for policy enforcement, but for the automated *inference* of a security specification from a codebase, providing a robust foundation for identifying intent-implementation mismatches.

**Definition 3.3 (Semantic Gap).** A semantic gap exists when: In this work,  $S$  denotes the implementation contained in a single source file, which is the current analysis scope of INTENTFIX (multi-file reasoning is left for future work).

$$\exists c \in SC, \exists s \in S : \neg \text{enforces}(s, c) \wedge \text{required}(c, \text{domain}(s))$$

where  $\text{enforces}(s, c)$  is a predicate evaluated by the LLM, checking if code segment  $s$  correctly implements constraint  $c$ .

**3.2.2 Intent-Centric Vulnerability Analysis.** Our approach builds on the empirically-supported hypothesis that large language models can recognize common programming patterns and security best practices from their training data [10, 48]. To distinguish between  $I_{\text{expected}}$  and  $I_{\text{actual}}$ , INTENTFIX employs distinct prompting strategies. For  $I_{\text{expected}}$ , we prompt: "As a Security Intent Analyst, identify intended security properties based on naming patterns, comments, domain conventions, and security best practices. Focus on what SHOULD be implemented." For  $I_{\text{actual}}$ , we prompt: "As a Code Behavior Analyzer, describe ONLY executed logic paths. Ignore names, comments, or missing functionality." This prompt-based separation enables systematic identification of semantic gaps between intended and actual behavior.

**Definition 3.4 (Logic Vulnerability as an Intent-Implementation Mismatch).** A code implementation  $S$  is defined to have a logic vulnerability with respect to an expected intent model  $I_{\text{expected}}$  if the actual intent model  $I_{\text{actual}}$  derived from  $S$  does not fully satisfy the expected model. Formally:

$$\text{has\_vulnerability}(S, I_{\text{expected}}) \iff I_{\text{actual}} \not\supseteq I_{\text{expected}}$$

where  $I_{\text{actual}} = \text{extract}(S)$  represents the intent model perfectly realized by the code. The core challenge, which this paper addresses, is prompting an LLM to robustly approximate both  $I_{\text{expected}}$  and  $I_{\text{actual}}$  from source code and its surrounding context.

### 3.3 Phase 1: LLM-Powered Intent Modeling

We assume that developer intentions are implicitly encoded in various code artifacts (variable names, function signatures, comments) and that LLMs, trained on vast code corpora, can recognize and extract these patterns. This assumption is validated empirically in Section 5.2.

**3.3.1 Hierarchical Intent Extraction.** We decompose intent extraction into four primary sub-tasks: Domain Analysis, Actor Identification, Resource Mapping, and Constraint Inference. This process, detailed in Algorithm 1, is not a traditional algorithm but a hierarchical prompt chain, where the structured output from one LLM query serves as critical context for the next. For example, the 'domain' model extracted in Step 1 is explicitly injected into the prompt for 'actor' identification in Step 2. This ensures each step of the analysis is progressively enriched with context, leading to a more coherent and accurate final intent model.

The key functions in Algorithm 1 operate as follows:

---

#### Algorithm 1: Hierarchical Intent Extraction Pipeline

---

**Input:** Source code  $S$ , Context  $C$ , LLM  $M$ , Confidence threshold  $\theta$

**Output:** Intent model  $I$

// Domain Analysis

1  $\text{prompt}_D \leftarrow \text{BuildDomainPrompt}(S, C)$ ;

2  $D_{\text{json}} \leftarrow \text{QueryLLM}(M, \text{prompt}_D, \text{role}="Domain Analyst")$ ;

3  $D \leftarrow \text{ParseAndValidateSchema}(D_{\text{json}}, \text{DomainSchema})$ ;

// Actor Identification

4  $\text{prompt}_A \leftarrow \text{BuildActorPrompt}(S, D)$ ;

5  $A_{\text{json}} \leftarrow \text{QueryLLM}(M, \text{prompt}_A, \text{role}="Security Architect")$ ;

6  $A \leftarrow \text{ParseAndValidateSchema}(A_{\text{json}}, \text{ActorSchema})$ ;

// Resource Mapping

7  $\text{prompt}_R \leftarrow \text{BuildResourcePrompt}(S, D, A)$ ;

8  $\{R_{\text{json}}, P_{\text{initial\_json}}\} \leftarrow \text{QueryLLM}(M, \text{prompt}_R, \text{role}="Access Control Expert")$ ;

// Constraint Inference

9  $\text{prompt}_{SC} \leftarrow \text{BuildConstraintPrompt}(S, D, A, R_{\text{json}})$ ;

10  $SC_{\text{raw}} \leftarrow \text{QueryLLM}(M, \text{prompt}_{SC}, \text{role}="Security Analyst")$ ;

11  $SC \leftarrow \text{FilterByConfidence}(SC_{\text{raw}}, \theta)$ ;

// Cross-validation and Assembly

12  $P \leftarrow \text{DerivePermissions}(A, R_{\text{json}}, P_{\text{initial\_json}}, SC)$ ;

13  $I \leftarrow \text{AssembleModel}(D, A, R_{\text{json}}, P, SC)$ ;

14 **return**  $I$ ;

---

- **BuildXPrompt:** Constructs structured prompts using templates (see Listing 1) with role-specific instructions and JSON schema constraints.
- **QueryLLM:** Sends the prompt to the LLM with temperature=0, validates the JSON response, and retries up to 3 times if schema validation fails.
- **ParseAndValidateSchema:** Parses the LLM's JSON response and validates it against a predefined schema. On failure, it can trigger a self-correction loop by re-prompting with the error.
- **FilterByConfidence:** Retains only constraints with confidence scores above threshold  $\theta$  (default: 0.8).

**3.3.2 Structured LLM Interaction Protocol.** The 'QueryLLM' function in our algorithms encapsulates a robust protocol of advanced prompt engineering techniques to ensure the reliability and verifiability of the LLM's output. This protocol is the technical core that enables INTENTFIX's performance.

**Step-by-Step Reasoning Priming.** To mitigate the risk of the LLM jumping to conclusions, every prompt begins by instructing the model to "think step-by-step," followed by a structured analytical framework. This technique, a form of Chain-of-Thought (CoT) priming [41], forces the LLM to follow a deliberate reasoning path, significantly improving its ability to handle complex code analysis tasks.

**Listing 1: Domain Analysis Prompt Template**

You are an expert software engineer **and** domain analyst.  
Your task is ...

Let's think step by step to thoroughly understand the domain:

## Step 1: Initial Code Examination  
First, let me examine the code structure and patterns:  
- What imports and dependencies are used?  
...

Code to analyze: {source\_code}  
```json  
{  
 "domain": "Brief, specific description of the  
 application domain",  
 "context": "Detailed explanation of the business  
 logic, main functionality, and key processes",  
 "security\_relevance": "Why this domain is important  
 for security analysis, including typical  
 vulnerabilities and attack vectors",  
 "confidence": 0.95, ...  
}

*Role-Based Persona Prompting.* As shown in our prompts, each query is assigned a specific 'role' (e.g., "Security Architect"). This primes the LLM to activate the most relevant knowledge and reasoning patterns for the sub-task at hand. A sample prompt template for domain analysis is shown in Listing 1. The full prompt templates, including step-by-step reasoning instructions and JSON schema constraints, are available in our online artifact.

**3.3.3 Intent Model Validation and Refinement.** Ensuring the quality of the generated intent model is critical. Our protocol incorporates two key validation mechanisms.

*Schema-Constrained Generation and Validation.* All prompts mandate a JSON output, which is validated against a strict schema. If the output is malformed, our system initiates a self-correction loop, re-sending the prompt with the validation error. This significantly improves the reliability of structured data extraction.

*Confidence-Based Filtering and Iterative Refinement.* The LLM is prompted to include a 'confidence' score in its output. We use a configurable threshold  $\theta$  to filter out low-confidence inferences. Furthermore, INTENTFIX supports an iterative refinement loop. As evidenced by our prompt, a generated intent model can be fed back into the LLM with instructions for targeted improvement, particularly for enhancing the quality of Semantic Constraints (SC).

**3.4 Phase 2: Semantic Gap Detection**

In this phase, INTENTFIX identifies vulnerabilities by prompting the LLM to perform a differential analysis between the high-level security requirements in the intent model and the source code.

**3.4.1 Differential Analysis Algorithm.** The process, outlined in Algorithm 2, systematically checks if each semantic constraint ( $c \in I.SC$ ) is enforced. For each constraint, the framework prompts the LLM to:

**Algorithm 2: LLM-Driven Semantic Gap Detection**


---

**Input:** Intent model  $I$ , Source code  $S$ , LLM  $M$   
**Output:** Vulnerabilities  $V$ , Attack Vectors  $AV$

```

1  $V \leftarrow \emptyset, AV \leftarrow \emptyset;$ 
2 foreach constraint  $c \in I.SC$  do
3    $prompt_{loc} \leftarrow \text{BuildLocatePrompt}(c, S);$ 
   // LLM finds relevant code
4    $impl \leftarrow \text{QueryLLM}(M, prompt_{loc});$ 
5    $v \leftarrow \text{null};$ 
6   if  $impl = \emptyset$  then
7      $v \leftarrow \text{CreateVuln}(c, \text{"MISSING"}, \emptyset);$ 
8   else
9      $prompt_{check} \leftarrow \text{BuildCheckPrompt}(c, impl);$ 
    // LLM checks logic
10     $isSatisfied \leftarrow \text{QueryLLM}(M, prompt_{check});$ 
11    if  $\neg isSatisfied$  then
12       $v \leftarrow \text{CreateVuln}(c, \text{"INCORRECT"}, impl);$ 
13    end
14  end
15  if  $v \neq \text{null}$  then
16     $V \leftarrow V \cup \{v\};$ 
17     $prompt_{av} \leftarrow \text{BuildAttackVectorPrompt}(v, S);$ 
    // LLM generates attack
18     $vector \leftarrow \text{QueryLLM}(M, prompt_{av});$ 
19     $AV \leftarrow AV \cup \{vector\};$ 
20  end
21 end
22 return  $(V, AV);$ 
```

---

- (1) **Locate Implementation:** Find the specific code segment responsible for enforcing the constraint.
- (2) **Check Satisfaction:** Analyze the located code to determine if it correctly and completely implements the logic of the constraint.

If the LLM reports that no implementation exists, or that the existing code is flawed, a potential vulnerability is flagged. This process transforms vulnerability discovery from an open-ended problem into a series of focused, verifiable constraint-checking queries.

**3.4.2 Attack Vector Generation.** For each detected semantic gap, generating a fully executable Proof-of-Concept (PoC) exploit is often impractical due to complex environmental dependencies (e.g., database state, external services, specific library versions). Instead, INTENTFIX adopts a more robust and generalizable approach: it prompts the LLM to generate a structured, natural-language attack vector. This vector serves two critical purposes: (1) it validates the identified vulnerability by articulating a concrete exploitation path, and (2) it provides crucial context for the subsequent patch generation phase.

This natural-language description precisely outlines the preconditions, step-by-step actions, and expected impact of an attack, serving as a "conceptual PoC" that is more resilient to environmental variations. Listing 2 shows an example of a generated attack vector for a path traversal vulnerability.

**Algorithm 3: LLM-Based Patch Refinement Loop**


---

**Input:** Vulnerable code  $S$ , Intent model  $I$ , Vulnerability info  $V_{info}$ , LLM  $M$   
**Output:** Refined patch  $P^*$  or failure  
 // Prompt LLM for first patch  
 1  $P_{candidate} \leftarrow \text{GenerateInitialPatch}(S, I, V_{info}, M)$ ;  
 // Prompt LLM to act as reviewer  
 2  $critique \leftarrow \text{AssessPatchWithLLM}(P_{candidate}, S, I, M)$ ;  
 3 **if**  $\text{IsCorrect}(critique)$  **then**  
 4     **return**  $P_{candidate}$ ;  
 5 **else**  
 6     // Re-prompt with critique  
 7      $P_{refined} \leftarrow \text{RefineWithFeedback}(S, I, V_{info}, critique, M)$ ;  
 8     // LLM re-assesses  
 9      $critique_{refined} \leftarrow \text{AssessPatchWithLLM}(P_{refined}, S, I, M)$ ;  
 10     **if**  $\text{IsCorrect}(critique_{refined})$  **then**  
 11         **return**  $P_{refined}$ ;  
 12     **end**  
 13 **end**  
 // Self-correction failed  
 14 **return** *failure*;

---

**Listing 2: Example of Attack Vector**

```
"gap_description": "Path Traversal and Arbitrary File
Access: The cacheDriver.PutBlob method is called
with file paths as values. Without proper
validation and sanitization of these file paths,
this may allow path traversal attacks or arbitrary
file access. ..."

"attack_scenario": "An attacker could exploit the
PutBlob method by passing specially crafted file
paths containing '../' sequences or absolute paths
to overwrite or read sensitive files outside the
intended cache directory. ..."
```

**3.5 Phase 3: LLM-Guided Patch Synthesis and Refinement**

The final phase generates a patch by guiding an LLM with a rich context, including the intent model and semantic gap analysis. Instead of relying on external verifiers, this phase uses the LLM's own reasoning capabilities in a self-correction loop.

A patch  $P$  must meet three goals, which are given as explicit directives in the prompt:

- **Security Adherence:** The patch must resolve the identified semantic gap ( $V$ ) and align with all security rules in the intent model ( $SC$ ).
- **Functionality Preservation:** The patch should not alter the original, intended functionality or break existing API contracts.
- **Code Quality:** The patch should be minimal, readable, and maintain the code style of the surrounding file.

**Table 1: Dataset Characteristics (N=1,107)**

| Metric                                   | Value       |
|------------------------------------------|-------------|
| Total vulnerability-patch pairs          | 1,107       |
| Average file size (LoC)                  | 615         |
| Average patch size (LoC)                 | 17.1        |
| <b>Top Vulnerability Types (CWE)</b>     |             |
| CWE-770 (Resource Allocation)            | 804 (72.6%) |
| CWE-639 (Authorization Bypass)           | 241 (21.8%) |
| CWE-640 (Weak Password Recovery)         | 52 (4.7%)   |
| Others (841, 283)                        | 10 (0.9%)   |
| <b>Top Programming Languages</b>         |             |
| PHP (20.7%), Go (14.5%), Python (13.7%), |             |
| C# (9.1%), Scala (9.6%), Ruby (7.7%)     |             |

**3.5.1 LLM-Based Self-Correction.** Generating a perfect patch in one shot is unlikely [5]. INTENTFIX employs an LLM-based self-correction loop, as detailed in Algorithm 3. If an initial patch is assessed as flawed, the system uses the LLM's own critique as feedback for refinement.

The 'AssessPatchWithLLM' function does not execute code. Instead, it queries the LLM again, providing the original code, the generated patch, and the intent model. It prompts the LLM to act as a code reviewer to critique the patch against the required constraints. This critique then serves as the "feedback" for the refinement step. This single-loop self-correction is highly effective at fixing common LLM-generated errors like minor syntax mistakes or incomplete logic.

**4 Experimental Setup**

We conducted comprehensive experiments to evaluate INTENTFIX's effectiveness in detecting and repairing real-world logic vulnerabilities. This section describes our research questions, dataset construction, baseline methods, and evaluation methodology.

**4.1 Research Questions**

Our evaluation addresses three research questions that systematically examine INTENTFIX's effectiveness, design choices, and practical applicability:

- **RQ1 (Effectiveness):** How effective is INTENTFIX compared to state-of-the-art approaches in automatically repairing logic vulnerabilities across different vulnerability types?
- **RQ2 (Component Analysis):** What is the contribution of each component (intent modeling, semantic gap detection, and multi-aspect refinement) to the overall effectiveness of INTENTFIX?
- **RQ3 (Scalability and Generalization):** To what extent does INTENTFIX scale to real-world software projects and generalize across different programming paradigms?

**4.2 Dataset Construction**

**4.2.1 Data Collection Pipeline.** We constructed a comprehensive dataset of logic vulnerabilities through a systematic four-stage process.

**Stage 1: CVE Collection** We started by collecting all CVE and CWE entries from the National Vulnerability Database [26, 36].

**Stage 2: Logic Vulnerability Filtering** We filtered the collected CVEs to target logic vulnerabilities. Specifically, we focused on CVEs linked to the CWE-840 (Business Logic Errors) category and its descendant CWEs (CWE-283, 639, 640, 708, 770, 826, 837, 841) [35]. This ensures our dataset is centered on the intended scope of logic flaws.

**Stage 3: Commit Mining** For each filtered CVE, we identified the corresponding fixing commit on GitHub. This was achieved by parsing the reference links provided in the CVE entries, which often point directly to commits, pull requests, or security advisories.

**Stage 4: Quality Filtering and Parsing** We collected the source code files before and after the fixing commit. To ensure data quality and suitability for analysis, we applied a final filtering step, retaining only those file pairs that could be successfully parsed into Abstract Syntax Trees (ASTs). This process yielded our final dataset of 1,107 high-quality vulnerability-patch pairs.

**4.2.2 Dataset Characteristics.** The final dataset comprises 1,107 high-quality vulnerability-patch pairs. Table 1 shows the distribution of vulnerability types, and provides detailed statistics.

As shown in Table 1, our dataset exhibits an imbalance, with CWE-770 (Allocation of Resources Without Limits) accounting for a significant portion (72.6%). This reflects the prevalence of this vulnerability type in real-world CVEs reported in the NVD. We acknowledge this skew and, to prevent biased interpretation of the results, we present our analysis not only on the overall performance but also broken down by CWE category in Section 5.1. This allows for a more nuanced understanding of INTENTFIX’s effectiveness across different types of logic vulnerabilities.

### 4.3 Baseline Methods

To evaluate INTENTFIX, we establish strong baselines using state-of-the-art LLMs rather than traditional test suite based APR tools. Recent work has shown that modern LLMs already match or outperform many classical APR systems on diverse bug types, and logic vulnerabilities often lack reliable test oracles, making test-driven repair less applicable in our setting [3, 8, 44, 49]. Zero-shot and Chain-of-Thought prompting therefore provide a strong and widely used reference point against which to measure the additional benefit of our intent-centric reasoning.

We use the following baseline configurations:

- **Zero-Shot LLM:** As our standard baseline, we use two leading models: OpenAI’s gpt-4.1-mini and Anthropic’s claude-3.7-sonnet. The model is provided with the vulnerable code snippet and a high-level description of the vulnerability. It is then prompted to generate a corrected version of the code in a single pass. This baseline measures the out-of-the-box repair capabilities of current LLMs.
- **Chain-of-Thought (CoT) LLM:** To create a more powerful baseline, we augment the zero-shot prompt with an instruction to “think step-by-step to analyze the vulnerability and devise a patch,” and provide guidelines. By comparing INTENTFIX against a CoT-prompted LLM, we can determine if

the observed performance gain stems from our novel intent-modeling methodology, rather than merely from employing a generic multi-step reasoning process.

### 4.4 Evaluation Metrics

We evaluate the performance of all approaches using a combination of automated assessment and manual expert validation to ensure a robust and reliable evaluation.

**4.4.1 Patch Correctness Assessment.** The core of our evaluation is determining whether a generated patch correctly fixes the target vulnerability without introducing regressions. Given the semantic nature of logic vulnerabilities, simple textual or syntactic checks are insufficient. We therefore employ a two-stage assessment process:

**Manual Expert Validation:** To validate our AI oracle and establish a definitive ground truth, a panel of three security experts manually evaluated patches. A patch was confirmed as ‘correct’ by a majority vote (at least 2 of 3 experts), with the experts achieving high inter-rater reliability (Fleiss’s kappa = 0.87). Our validation was rigorous: for our main effectiveness claim (RQ1), we manually reviewed all patches that the AI oracle marked as correct. For the component analysis (RQ2), we assessed a stratified random sample of 300 patches. This manual verification provides a solid foundation for the key claims of our work.

**AI-based Assessment:** To robustly evaluate patch correctness at scale, we employed an AI-based oracle using OpenAI’s o4-mini. For each case, the oracle was provided with the vulnerable code, the generated patch, the ground-truth fix, and the CWE description. It was then tasked with judging correctness based on two criteria: (1) Does the patch fix the specified vulnerability? (2) Does it preserve the original functionality and introduce no new bugs?

**4.4.2 Statistical Significance Testing.** To rigorously validate our claims of improvement, we perform statistical significance testing for all primary comparisons. Since we compare the performance of different approaches on the same set of vulnerabilities, the data is paired. We use the McNemar’s test [22], which is appropriate for comparing the accuracy of two classifiers on paired nominal data. We report p-values for our key results, considering  $p < 0.05$  as statistically significant.

### 4.5 Experimental Protocol

All experiments were performed on a workstation with the following specifications:

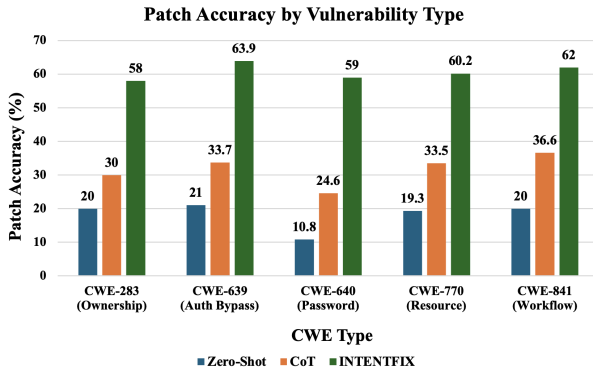
- **Hardware:** Intel Core i9-12900K CPU with 64GB RAM.
- **Software:** Python 3.9, OpenAI API library version 1.66.0.
- **LLM Settings:** For all model interactions (both for repair and evaluation), the temperature parameter was set to 0.0 to ensure deterministic and reproducible outputs. All other hyperparameters were kept at their default values.

## 5 Experimental Results

Our experiments are designed to rigorously answer the research questions outlined in Section 4.1. We first present the overall effectiveness of INTENTFIX compared to baselines, then delve into a component-wise ablation study, and finally analyze its scalability and generalization capabilities.

**Table 2: Overall Patch Accuracy Comparison**

| Approach                            | Generated Patch | Correct Patch(AI Eval)   | Correct Patch(Manual Eval) |
|-------------------------------------|-----------------|--------------------------|----------------------------|
| Zero-shot(GPT-4.1-Mini)             | 1107/1107       | 409 / 1107(36.9%)        | 287 / 1107(25.9%)          |
| CoT(GPT-4.1-Mini)                   | 1107/1107       | 459 / 1107(41.4%)        | 362/1107(32.7%)            |
| <b>INTENTFIX(GPT-4.1-Mini)</b>      | 1029/1107       | <b>857 / 1107(77.4%)</b> | <b>714/1107(64.5%)</b>     |
| Improvement over baseline           |                 | <b>+36.0% (1.86×)</b>    | <b>+31.8% (1.97×)</b>      |
| Zero-shot(Claude 3.7 Sonnet)        | 1107/1107       | 520 / 1107 (47.0%)       | 331/1107(29.9%)            |
| CoT(Claude 3.7 Sonnet)              | 1107/1107       | 579 / 1107 (52.3%)       | 432/1107(39.0%)            |
| <b>INTENTFIX(Claude 3.7 Sonnet)</b> | 1037/1107       | <b>893 / 1107(80.6%)</b> | <b>734/1107(66.3%)</b>     |
| Improvement over baseline           |                 | <b>+28.3% (1.54×)</b>    | <b>+27.3% (1.7×)</b>       |

**Figure 2: Patch Accuracy by Vulnerability Type**

## 5.1 RQ1: Effectiveness of Intent-Centric Repair

To answer RQ1, we evaluated INTENTFIX against strong baselines on our comprehensive dataset of 1,107 real-world CVEs.

**5.1.1 Overall Performance.** As shown in Table 2, our primary evaluation, based on rigorous manual validation by security experts, reveals that INTENTFIX achieves a patch accuracy of 64.5%. This represents a 1.97× improvement (from 32.7% to 64.5%, +31.8 percentage points) over the CoT baseline’s manually-verified accuracy. A McNemar’s test confirms this improvement is statistically significant ( $p < 0.001$ ). While the AI-based evaluation shows INTENTFIX achieving 77.4% accuracy, manual validation reveals a more conservative 64.5%. This 12.9 percentage point gap suggests the AI oracle may overestimate correctness, potentially due to stylistic similarity bias. Despite this limitation, both evaluations consistently show INTENTFIX outperforming baselines by a significant margin.

This substantial margin highlights a key finding: for complex logic vulnerabilities, simply prompting an LLM (even with CoT) is insufficient. The structured, multi-phase approach of INTENTFIX, which first builds an explicit model of developer intent, provides the necessary semantic context for the LLM to generate correct fixes, rather than plausible but flawed patches.

**5.1.2 Performance by Vulnerability Type.** Figure 2 breaks down the accuracy by CWE category. INTENTFIX consistently outperforms

### Listing 3: CVE-2021-3964: Vulnerable Code and INTENTFIX-Generated Patch

```
// Vulnerable code
$user_guid = (int) get_input('user_guid',
    elgg_get_logged_in_user_guid());
$user = get_user($user_guid);
$group = get_entity((int) get_input('group_guid'));
// Missing: authorization check
if (!$group->leave($user)) { return error; }
// INTENTFIX-generated patch
if ($user->guid !== $logged_in_user->guid) {
    if (!$group->canEdit()) return error;
}
```

baselines across all types. Its advantage is most pronounced in CWE-639 (Authorization Bypass,  $n=241$ ), achieving 63.9% accuracy. For authorization-related vulnerabilities like CWE-639, understanding the intended workflow and authorization rules—the core function of our intent model—is critical for a correct fix. CWE-841 (Improper Behavioral Workflow) shows 62.0% accuracy, though this result should be interpreted cautiously due to the small sample size ( $n=10$ ). While the baseline models show some capability in fixing more direct issues like CWE-770 (Resource Allocation), they struggle significantly with complex authorization and workflow logic, where INTENTFIX’s structured analysis excels.

**5.1.3 Performance by Programming Language.** To assess the generalizability of our framework, we analyzed its performance across the 19 programming languages in our dataset. Due to space constraints, Table 3 presents a summary, highlighting the top three languages by sample count for both statically and dynamically-typed categories, while aggregating the remaining 13 languages.

The results clearly demonstrate INTENTFIX’s broad effectiveness. It consistently and substantially outperforms the CoT baseline across all major languages, achieving high accuracy in widely-used languages like Go (73.7%), PHP (67.0%), and Python (64.6%). Crucially, the strong performance on the aggregated ‘Other Languages’ category (62.2% accuracy on 273 samples) provides compelling evidence that our method is not overfitted to a few specific languages.

Furthermore, the overall accuracy for statically-typed languages (65.6%) and dynamically-typed languages (62.7%) is nearly identical. This parity suggests that our approach successfully abstracts

**Table 3: Performance Across Top Programming Languages by Sample Count. Results for the remaining 13 languages are aggregated into the 'Other Languages' category.**

| Language (Category)                    | Count       | CoT          | INTENTFIX    |
|----------------------------------------|-------------|--------------|--------------|
| <i>Top Statically-Typed Languages</i>  |             |              |              |
| Go                                     | 161         | 34.1%        | 73.7%        |
| Scala                                  | 106         | 25.8%        | 57.8%        |
| C#                                     | 101         | 23.7%        | 55.3%        |
| <i>Top Dynamically-Typed Languages</i> |             |              |              |
| PHP                                    | 229         | 38.9%        | 67.0%        |
| Python                                 | 152         | 27.5%        | 64.6%        |
| Ruby                                   | 85          | 27.6%        | 61.9%        |
| Other Languages (13)                   | 273         | 30.2%        | 62.2%        |
| <b>Overall</b>                         | <b>1107</b> | <b>32.7%</b> | <b>64.5%</b> |

semantic intent away from language-specific syntactic differences, confirming its robustness and wide applicability.

**5.1.4 Illustrative Case Study.** To concretely demonstrate how INTENTFIX’s intent modeling drives vulnerability repair, we present an analysis of CVE-2021-3964, an authorization bypass vulnerability in a social networking platform’s group management module. Listing 3 shows a simplified excerpt of the vulnerable code.

The code accepts a `user_guid` parameter from user input without validating that the requester has authorization to act on behalf of that user. An attacker can force any user to leave any group by simply specifying an arbitrary `user_guid`.

**Extracted Intent Model.** From this code, INTENTFIX extracted the following 5-tuple intent model:

- **D (Domain):** Online community group management
- **A (Actors):** Group Owner, Group Member, Logged-in User, Anonymous User, System Actor
- **R (Resources):** User Entity, Group Entity, Leave Group API Endpoint
- **SC (Key Constraint):** “Verify that the logged-in user’s GUID matches the provided `user_guid`, or the caller has administrative privileges”

**Gap Detection and Patch Synthesis.** The semantic constraint directly identifies the missing authorization check. INTENTFIX generated the patch shown in Listing 3, which enforces that users can only remove themselves unless they have group management privileges. The developer’s actual fix uses an equivalent check: `if (!$user->canEdit() && !$group->canEdit())`. Both patches achieve the same security goal—preventing unauthorized forced removal—demonstrating that explicit intent modeling enables semantically correct repairs.

**5.1.5 Qualitative Failure Analysis.** To understand INTENTFIX’s limitations, we systematically analyzed 100 randomly sampled failure cases from the 393 incorrect patches. Three researchers independently categorized failure patterns, achieving substantial agreement (Fleiss’s  $\kappa = 0.81$ ) [7].

**Table 4: Ablation Study Results**

| Configuration                         | AI Eval.     | Manual Eval.* |
|---------------------------------------|--------------|---------------|
| <b>INTENTFIX (Full)</b>               | <b>77.4%</b> | <b>64.5%</b>  |
| <i>Remove Intent Modeling</i>         |              |               |
| Direct vulnerability detection        | 64.5%        | 48.8%         |
| <i>Remove Semantic Gap Detection</i>  |              |               |
| Pattern-based detection only          | 60.1%        | 45.3%         |
| <i>Remove Multi-Aspect refinement</i> |              |               |
| Single-pass patch generation          | 63.1%        | 50.2%         |
| <i>Simplified Intent Model</i>        |              |               |
| Without Domain (D)                    | 63.1%        | 56.4%         |
| Without Actors (A)                    | 62.8%        | 54.3%         |
| Without Resources (R)                 | 62.1%        | 55.7%         |
| Without Permissions (P)               | 63.6%        | 56.5%         |
| Without Constraints (SC)              | 62.3%        | 53.4%         |

- (1) **Complex Multi-File Dependencies:** Vulnerabilities where the developer’s full intent was distributed across multiple files and complex call chains [19]. Our current single-file analysis context was insufficient to capture the complete logic.
- (2) **Highly Domain-Specific Logic:** Cases involving niche business domains (e.g., specific scientific computing or financial protocols) where the LLM lacked the specialized knowledge to understand the required constraints.
- (3) **Incorrect Intent Modeling:** In a few cases, the intent model itself was flawed, particularly in actor extraction, leading the subsequent repair phases to fix a non-existent or incorrect vulnerability.

These cases highlight important avenues for future work, such as inter-procedural analysis and domain-specific fine-tuning.

**Answer to RQ1.** INTENTFIX is highly effective. It achieves a patch correctness of **77.4% (AI Eval)** and **64.5% (Manual Eval)**, representing a statistically significant 1.86x improvement over the state-of-the-art CoT baseline ( $p < 0.001$ ). Its performance is robust across diverse vulnerability types and programming languages, demonstrating that the intent-centric paradigm is a significant advancement for repairing logic vulnerabilities.

## 5.2 RQ2: Component Contribution Analysis

To understand the sources of INTENTFIX’s performance, we conducted an ablation study, systematically removing or simplifying key components of the framework. For manual evaluation, we randomly sampled 300 patches and manually validated them. With this sample size, we achieve a 95% confidence level with a margin of error of  $\pm 5.7\%$  for the reported proportions.

The results in Table 4 confirm that every component of INTENTFIX contributes significantly to its overall accuracy. Removing the

core Intent Modeling phase causes the most substantial drop in performance (from 64.5% to 48.8% in AI eval), confirming that explicit intent extraction is the most critical innovation of our framework. This performance degradation is statistically significant (McNemar’s test,  $p < 0.001$ ), providing strong evidence that explicit intent extraction is indeed the most critical innovation of our framework.

Similarly, removing the semantic gap detection module and relying only on pattern-based methods leads to a significant performance degradation (45.3% accuracy). This shows that our differential analysis is crucial for precisely pinpointing the mismatch between intent and implementation. The multi-aspect refinement loop also proves vital; removing it and relying on a single-pass generation drops accuracy to 50.2%, as it fails to catch subtle errors in the initial patch.

Furthermore, simplifying the 5-tuple intent model by removing individual components (D, A, R, P, or SC) consistently lowers accuracy. The removal of semantic constraints (SC) has the largest negative impact among the tuple elements, underscoring the importance of capturing domain-specific business rules, which are often the source of logic vulnerabilities. Additionally, we manually validated 300 bugs to verify the quality of intent modeling. This validation process followed the same methodology as the previous manual verification. We confirmed that intent modeling achieved 88.1% accuracy overall. However, while domain, resource, permission, and constraints were extracted with nearly 90% accuracy, actor extraction only achieved 60% accuracy. The lower accuracy for actor extraction is due to actors often being implicitly represented through authentication tokens, session variables, or API parameters rather than being explicitly declared in code. In contrast, resources and permissions are often explicitly declared through assignments and method calls, while domain and constraints operate at a higher abstraction level, making them relatively easier to extract.

**Answer to RQ2.** The intent-centric design is the primary driver of INTENTFIX’s success. The intent modeling phase is the most critical component, and the semantic gap detection and multi-aspect refinement phases provide further essential contributions. Each element of the 5-tuple intent model is proven to be valuable for achieving high accuracy.

### 5.3 RQ3: Scalability and Generalization

Finally, we assessed INTENTFIX’s performance on larger files and its ability to generalize to unseen vulnerabilities and standard benchmarks.

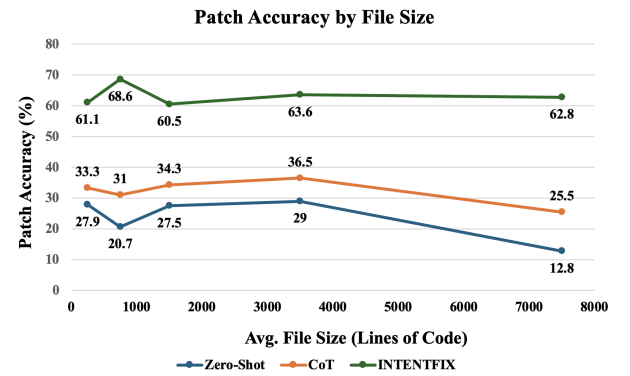
**5.3.1 Performance vs. Code Complexity.** Figure 3 shows the relationship between file size (LoC) and patch accuracy. Remarkably, INTENTFIX’s performance does not degrade for larger files, maintaining strong performance (e.g., 62.8% on files up to 10,000 LoC). This indicates that our hierarchical analysis effectively manages complexity. The average processing time and cost per file are 171.4 seconds and \$0.010 respectively (median: 89.2 seconds, \$0.006). For files exceeding 5,000 LoC, costs can reach up to \$0.089, though such cases represent only 8.3% of our dataset. A cost breakdown reveals the distribution of this expense across our framework’s phases: Intent Modeling, being the most semantically intensive,

**Table 5: Number of Correct Patches on Post-Cutoff Vulnerabilities (out of 115 total).**

| Metric          | Zero-shot    | CoT          | INTENTFIX   |
|-----------------|--------------|--------------|-------------|
| Generated Patch | 115 (100.0%) | 115 (100.0%) | 109 (94.8%) |
| AI eval         | 48 (41.7%)   | 57 (49.6%)   | 81 (70.4%)  |
| Manual eval     | 29 (25.2%)   | 38 (33.0%)   | 64 (55.7%)  |

**Table 6: Performance on VJBench**

| Metric        | Zero-shot  | CoT         | INTENTFIX   |
|---------------|------------|-------------|-------------|
| CodeBLEU      | 0.2056     | 0.2714      | 0.4356      |
| Correct Patch | 8/50 (16%) | 13/50 (26%) | 21/50 (42%) |



**Figure 3: INTENTFIX Patch Accuracy vs. File Size**

accounts for approximately 65% of the cost; Patch Synthesis and Refinement accounts for 25%; and the lighter-weight Gap Detection phase consumes the remaining 10%. This confirms the framework’s applicability to real-world projects.

**5.3.2 Generalization to Post-Cutoff Data.** To address potential data contamination threats, we tested our framework on 115 datasets, which are data after the knowledge cutoff date of gpt-4.1-mini, 01-06-2024 [28]. As shown in Table 5, INTENTFIX achieves 70.4% (AI eval) and 55.7% (Manual eval) on post-cutoff vulnerabilities. The 8.8 percentage point drop in manual evaluation accuracy (from 64.5% to 55.7%) compared to the full dataset suggests some dependency on patterns from the training period, though the framework still significantly outperforms baselines. Crucially, its performance remains significantly higher than the CoT baseline (49.6% AI eval) on this post-cutoff dataset (McNemar’s test,  $p < 0.001$ ). This result provides strong evidence that INTENTFIX’s success stems from its robust methodology, not from memorizing known vulnerability patterns from its training data.

**5.3.3 Apply to benchmark dataset.** We evaluated our approach on the VJBench [42] dataset to compare against other AVR techniques in a standardized setting. As presented in Table 6, INTENTFIX achieved a correctness of 42% (21 out of 50), which is a 1.62x improvement over the CoT baseline (26%) and a 2.63x improvement

over the zero-shot baseline (16%). The CodeBLEU [30] score also shows a similar trend. While the absolute accuracy is lower than on our CVE dataset, which is expected due to the different nature and complexity of benchmark tasks, the substantial relative improvement confirms the superiority of our intent-centric approach.

**Answer to RQ3.** INTENTFIX scales effectively to large, complex codebases without sacrificing accuracy. It successfully generalizes to unseen, post-cutoff vulnerabilities and outperforms baselines on standard benchmarks, confirming the robustness and broad applicability of our approach.

## 6 Discussion

### 6.1 Technical Limitations and Future Work

**(1) Single-File Analysis Scope:** INTENTFIX currently operates within single-file contexts, a design choice that ensures tractability and cost-effectiveness. While this approach proves effective for many vulnerabilities, extending to inter-procedural analysis across multiple files remains an important direction for future work. We envision a staged approach: first supporting direct imports and includes, then implementing lightweight call-graph analysis for security-critical paths.

**(2) Actor Extraction Accuracy:** Our intent modeling achieves 88.1% overall accuracy, but actor extraction lags at 60% due to the implicit representation of actors through authentication tokens and session variables. Despite this limitation, INTENTFIX maintains high overall effectiveness because many logic vulnerabilities can be detected through successfully extracted resources and permissions. Future work should develop specialized techniques for authentication pattern recognition and data-flow analysis to improve actor identification.

These limitations do not invalidate our core contributions but rather highlight promising directions for extending the intent-centric paradigm.

### 6.2 Threats to Validity

**Internal Validity.** The primary threat is the potential for data contamination, where CVEs from our dataset were present in the LLMs' training data. We mitigate this in two ways. First, our framework's significant outperformance of the zero-shot baseline suggests that INTENTFIX's structured process, not mere memorization, is the source of its strength. Second, and more importantly, our experiment on 115 post-cutoff CVEs shows that INTENTFIX maintains its high accuracy (55.7% overall success) on data guaranteed to be unseen, confirming its generalization capabilities.

**External Validity.** Our dataset, while large, is skewed towards web technologies (e.g., PHP, Python). Although INTENTFIX performed well across 19 languages, its effectiveness in other domains like embedded systems or scientific computing requires further investigation. Furthermore, our findings are specific to logic vulnerabilities (CWE-840 and descendants) and do not necessarily generalize to other bug classes like memory safety errors.

**Construct Validity.** A significant threat to construct validity is our reliance on an "AI oracle" for large-scale patch correctness assessment. While the "LLM-as-evaluator" paradigm [21, 32] is an

approach increasingly explored in recent literature for scalable evaluation, it carries an inherent risk of systemic bias. To mitigate this threat, our primary results (RQ1) are based on complete manual validation of all 1,107 patches, with the AI oracle serving only as a supplementary metric.

### 6.3 Data Availability and Reproducibility

To uphold the principles of open science and ensure the verifiability of our results, we have made all artifacts associated with this research publicly available. Our replication package includes: (1) the source code for the INTENTFIX framework, (2) our dataset of 1,107 vulnerability-patch pairs, (3) the scripts used to conduct all experiments and evaluations, and (4) the raw data generated from our results, and (5) the full prompts used for the AI oracle and the list of manually validated samples. The artifacts can be accessed at: <https://github.com/mrhjs225/intentfix-icse2026>.

### Acknowledgments

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No.RS-2024-00438686, Development of software reliability improvement technology through identification of abnormal open sources and automatic application of DevSecOps).

## 7 Conclusion

This paper introduced INTENTFIX, a framework that applies intent-centric security analysis to automatically repair logic vulnerabilities. By systematically leveraging LLMs to model developer intentions, identify semantic gaps, and synthesize verified patches, INTENTFIX achieves a patch accuracy of 77.4% (AI eval) and 64.5% (manual eval) on a large, real-world CVE dataset. This represents a 1.97x improvement over strong baselines, providing strong evidence that explicit intent modeling significantly enhances the effectiveness of automated vulnerability repair for logic flaws.

Our work not only provides a practical and effective tool but also establishes a new theoretical lens for understanding and resolving logic flaws. The future of automated security analysis lies not in simply reading code, but in understanding its purpose. Future research can build upon our work by integrating formal methods for stronger guarantees, developing interactive human-in-the-loop systems, and extending intent modeling to be a proactive tool that guides developers toward writing more secure code from the outset.

## References

- [1] Nicola Atzei, Massimo Bartoletti, and Tiziana Cimoli. 2017. A survey of attacks on ethereum smart contracts (sok). In *International conference on principles of security and trust*. Springer, 164–186.
- [2] Earl T Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2014. The oracle problem in software testing: A survey. *IEEE transactions on software engineering* 41, 5 (2014), 507–525.
- [3] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. Repairagent: An autonomous, llm-based agent for program repair. *arXiv preprint arXiv:2403.17134* (2024).
- [4] Quang-Cuong Bui, Ranindya Paramitha, Duc-Ly Vu, Fabio Massacci, and Riccardo Scandariato. 2024. APR4Vul: an empirical study of automatic program repair techniques on real-world Java vulnerabilities. *Empirical software engineering* 29, 1 (2024), 18.

- [5] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching Large Language Models to Self-Debug. *arXiv:2304.05128 [cs.CL]* <https://arxiv.org/abs/2304.05128>
- [6] Dongwook Choi and Eunseok Lee. 2025. Automated Feedback Generation for Programming Assignments Through Diversification. In *2025 IEEE/ACM 37th International Conference on Software Engineering Education and Training (CSEET)*. IEEE, 230–241.
- [7] Joseph L Fleiss. 1971. Measuring nominal scale agreement among many raters. *Psychological bulletin* 76, 5 (1971), 378.
- [8] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. 2022. VulRepair: a T5-based automated software vulnerability repair. In *Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering*. 935–947.
- [9] Jinseok Heo and Eunseok Lee. 2024. Analyzing the Impact of Context Representation and Scope in Code Infilling. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. 333–334.
- [10] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.
- [11] Vincent C Hu, David Ferraiolo, Richard Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. 2014. *Guide to attribute based access control (ABAC) definition and considerations*. NIST.
- [12] IBM. 2024. *Cost of a Data Breach Report 2024*. Technical Report. IBM. Accessed: 2025-07-19.
- [13] Imperva Threat Research. 2024. *The State of API Security in 2024*. Technical Report. Imperva. <https://www.imperva.com/resources/resource-library/reports/the-state-of-api-security-in-2024/>
- [14] Michael Jackson. 2000. *Problem Frames: Analyzing and structuring software development problems*. Addison-Wesley Longman Publishing Co., Inc.
- [15] Jensec. 2022. How a Business Logic Vulnerability Led to Unlimited Discount Redemption. <https://www.hackerrone.com/blog/how-business-logic-vulnerability-led-unlimited-discount-redemption>. Accessed: 2025-07-19.
- [16] Nan Jiang, Xiaopeng Li, Shiqi Wang, Qiang Zhou, Sonaya B Hossain, Baishakhi Ray, Varun Kumar, Xiaoqi Ma, and Anoop Deoras. 2024. Ledex: Training LLMs to better self-debug and explain code. *Advances in Neural Information Processing Systems* 37 (2024), 35517–35543.
- [17] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. Inferfix: End-to-end program repair with llms. In *Proceedings of the 31st ACM joint european software engineering conference and symposium on the foundations of software engineering*. 1646–1656.
- [18] Kyungho Lee and Misoo Kim. 2025. Zero-Shot LLM-based Automated Malicious Code Repair and Evaluation. *Korean Information Science Society Academic Conference Proceedings* (2025), 1949–1951.
- [19] Ying Li, Faysal Hossain Shezan, Bomin Wei, Gang Wang, and Yuan Tian. 2025. Sok: Towards effective automated vulnerability repair. *arXiv preprint arXiv:2501.18820* (2025).
- [20] Zongwei Li, Xiaoqi Li, Wenkai Li, and Xin Wang. 2025. SCALM: Detecting Bad Practices in Smart Contracts Through LLMs. *arXiv:2502.04347 [cs.CL]* <https://arxiv.org/abs/2502.04347>
- [21] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-eval: NLG evaluation using gpt-4 with better human alignment. *arXiv preprint arXiv:2303.16634* (2023).
- [22] Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12, 2 (1947), 153–157.
- [23] Bilgin Metin, Martin Wynn, Aylin Tunali, and Yağmur Kepir. 2025. Business Logic Vulnerabilities in the Digital Era: A Detection Framework Using Artificial Intelligence. *Information* 16, 7 (2025), 585.
- [24] Microsoft. 2024. *Microsoft Digital Defense Report 2024*. Technical Report. Microsoft. Accessed: 2025-07-19.
- [25] MITRE Corporation. 2014. CWE-840: Use of Max-Privilege Application Execution Level. Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/840.html>
- [26] National Institute of Standards and Technology. 2025. National Vulnerability Database (NVD) - Vulnerabilities API. <https://nvd.nist.gov/developers/vulnerabilities>. Accessed: 2025-07-19.
- [27] Yu Nong, Mohammed Aldeen, Long Cheng, Hongxin Hu, Feng Chen, and Haipeng Cai. 2024. Chain-of-thought prompting of large language models for discovering and fixing software vulnerabilities. *arXiv preprint arXiv:2402.17230* (2024).
- [28] OpenAI. 2025. Models - OpenAI API. <https://platform.openai.com/docs/models>. Accessed: 2025-07-19.
- [29] OWASP Foundation. 2021. *OWASP Top Ten*. <https://owasp.org/www-project-top-ten/>
- [30] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* (2020).
- [31] Haifeng Ruan, Yuntong Zhang, and Abhik Roychoudhury. 2024. Specrover: Code intent extraction via llms. *arXiv preprint arXiv:2408.02232* (2024).
- [32] Lin Shi, Chiyu Ma, Wenhua Liang, Xingjian Diao, Weicheng Ma, and Soroush Vosoughi. 2024. Judging the judges: A systematic study of position bias in llm-as-a-judge. *arXiv preprint arXiv:2406.07791* (2024).
- [33] Adam Shostack. 2014. *Threat modeling: Designing for security*. John Wiley & Sons.
- [34] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Haijun Wang, Zhengzi Xu, Xiaofei Xie, and Yang Liu. 2024. Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [35] The MITRE Corporation. 2024. CWE-840: Business Logic Errors. <https://cwe.mitre.org/data/definitions/840.html>. Accessed: 2025-07-19.
- [36] The MITRE Corporation. 2025. Common Weakness Enumeration (CWE) Downloads. <https://cwe.mitre.org/data/downloads.html>. Accessed: 2025-07-19.
- [37] Tony UcedaVelez and Marco M Morana. 2015. *Risk centric threat modeling: Process for attack simulation and threat analysis (PASTA)*. John Wiley & Sons.
- [38] Axel Van Lamsweerde. 2001. Goal-oriented requirements engineering: A guided tour. In *Proceedings fifth ieee international symposium on requirements engineering*. IEEE, 249–262.
- [39] Chong Wang, Jianan Liu, Xin Peng, Yang Liu, and Yiling Lou. 2025. Boosting static resource leak detection via llm-based resource-oriented intention inference. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 668–668.
- [40] Jiayimei Wang, Tao Ni, Wei-Bin Lee, and Qingchuan Zhao. 2025. A contemporary survey of large language model assisted program analysis. *arXiv preprint arXiv:2502.18474* (2025).
- [41] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [42] Yi Wu, Nan Jiang, Hung Viet Pham, Thibaud Lutellier, Jordan Davis, Lin Tan, Petr Babkin, and Sameena Shah. 2023. How effective are neural networks for fixing security vulnerabilities. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1282–1294.
- [43] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1482–1494.
- [44] Boyang Yang, Zijian Cai, Fengling Liu, Bach Le, Lingming Zhang, Tegawendé F Bissyandé, Yang Liu, and Haoye Tian. 2025. A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications. *arXiv preprint arXiv:2506.23749* (2025).
- [45] Hang Yuan, Lei Yu, Zhirong Huang, Jingyuan Zhang, Junyi Lu, Shiqi Cheng, Li Yang, Fengjun Zhang, Jiajia Ma, and Chun Zuo. 2025. Mos: Towards effective smart contract vulnerability detection through mixture-of-experts tuning of large language models. *arXiv preprint arXiv:2504.12234* (2025).
- [46] Silvan Zeller, Narges Khakpour, Danny Weyns, and Daniel Deogun. 2020. Self-protection against business logic vulnerabilities. In *Proceedings of the IEEE/ACM 15th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 174–180.
- [47] Zhuo Zhang, Brian Zhang, Wen Xu, and Zhiqiang Lin. 2023. Demystifying exploitable bugs in smart contracts. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 615–627.
- [48] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. 2025. Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–31.
- [49] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. 2025. Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–31.
- [50] Xin Zhou, Kisub Kim, Bowen Xu, DongGyun Han, and David Lo. 2024. Out of sight, out of mind: Better automatic vulnerability repair by broadening input ranges and sources. In *Proceedings of the IEEE/ACM 46th international conference on software engineering*. 1–13.